Diamond Software Documentation

Version SNAPSHOT

Table of Contents

If you want to go fast and far in a sustainable way, you need to go slow and lightweight with your friends.

Chapter 1. Introduction

1.1. Purpose

TODO

1.2. Decision Documentation

We use ADRs (Architecture Decision Records) to document important architectural and technical decisions made during the development of this software project.

For more explanation what ADRs are, please read: https://github.com/seepick/diamond/blob/main/doc/decisions/README.adoc

Chapter 2. Architecture

2.1. Service Types

- single codebase, or multiple (classpath isolated) codebases
- single service (pseudo-monolith) vs multiple services (isolation, performance)
- TODO read and summarize: https://nordicapis.com/whats-the-difference-microservice-macroservice-monolith/

2.1.1. Monolith

- one codebase, one deployable
- GOOD: low complexity (versioning, service location) and high speed (calls via code in same process instead network)
- BAD: change-resistant, slow, …

2.1.2. Modulith

- one codebase, basically just a monolith but a bit cleaner packaging; domain wise, not tech
- all the disadvantages of a monolith
- degrees:
- low version: only java packages (enforce package import; use spring modulith)
- mid version: maven sub-modules (sub-projects in gradle) or external repos with dependency on them
- high version: classpath isolated libraries (OSGi bundles, plugin architecture; extreme low coupling)
- this seems to be the only one counteracting the monolith's nature of low/mid version

2.1.3. Macroservices

- bigger sized microservices
- a bit less (manageable) complexity (pros of monolith) and still good enough to gain from the benefits (pros of modulith)

2.1.4. Microservices

- one very small codebase to one very small deployable
- GOOD:
- one service down, rest of application still runs
- very quick build time; code very isolated, small, thus easy to change
- BAD:

- complexity, service orchestration, versioning

2.1.5. Nanoservices

- deploy functions (AWS lambda)
- seems too extreme; complete different approach
- https://medium.com/@shrinit.poojary12/microservices-vs-nano-services-a-detailed-comparison-993453e65b8c

2.2. Layering

- Each architectural layer is represented by a Gradle sub-project (or sub-module as called in Maven terminology).

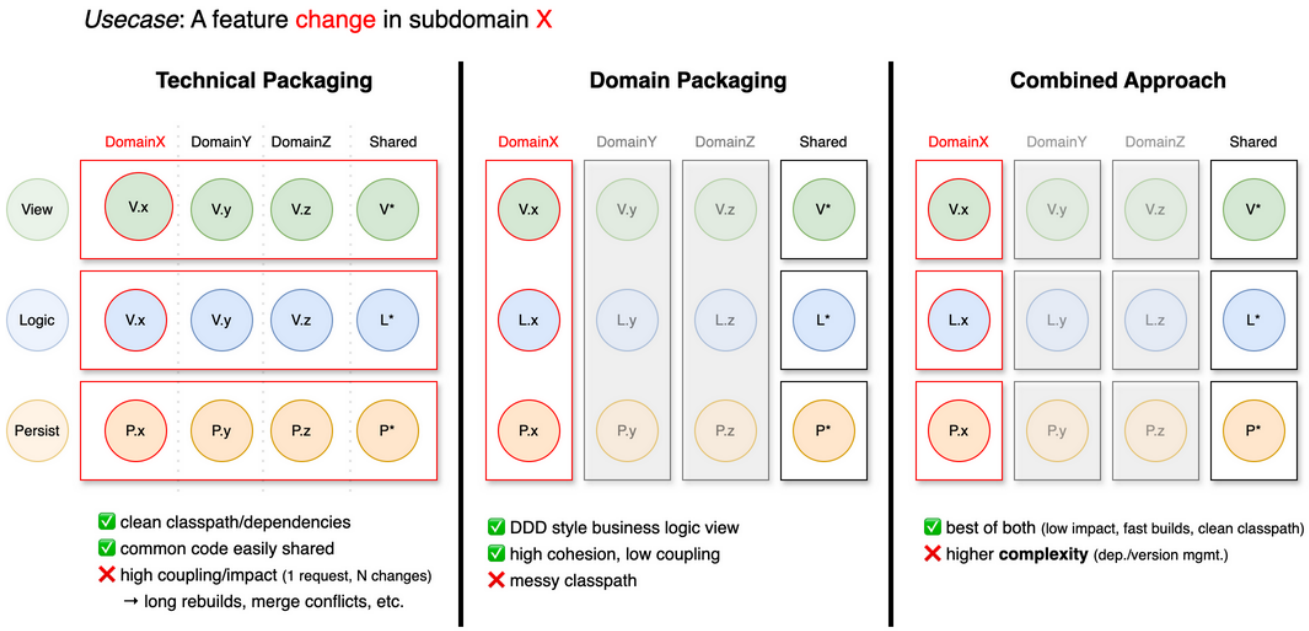

Figure 1. The degree of impact on the application depending on the choice of packaging strategy

Main Subprojects:

- app
- view
  - view-routing and view-controller-[api/impl] could be merged into one…
- logic-api
- logic-impl
- extern
  - extern-api
  - extern-impl (HTTP, SFTP)
  - extern-stub
  - extern-model (?) - generated source code (YAML, WSDL)

- persistence
  - persistence-api
  - persistence-impl (is actually persistence-exposed)
  - once there was a persistence-stub, but it turned out to be useless (already in full control)

Other Subprojects:

- client-sdk (kotlin MP)
  - sdk
  - models
  - tests?
- openapi-gen (resides in shared)

Test Subprojects:

- itest
- etest: separate GIT repo (maybe dep on client-model)

Shared Subprojects:

- test
- logging

2.2.1. ADR - Language Choice

- Status: Draft
- Created: 5.12.2025
- Author: Seepick

> Java or Kotlin?

TODO

Definitely no Scala, Clojure, ... and no non-JVM languages (Go, Rust, Python, C#, ...).

2.2.2. ADR - Webframework Choice

- Status: Draft
- Created: 5.12.2025
- Author: Seepick

> Spring Boot or Ktor? (Quarkus?)

Context:

- besides people dont know it, less mature, less doc
- much needs handwritten (openapi generation, jpa orm)
- the thing with spring boot is, very much like maven, “super strict, either our or no way”; very opinionated (convention over configuration like)
  - it’s beneficial, speed up, and stuff, but once you want to do more custom made things, it’s a fucking pain in the ass (like openshift: you are a pro user, use k8s directly)
- if you want to start only certain parts of the application in an isolated way, and then also in parallel, then spring boot is impossible for that
  - trust me, i tried it several times. it’s fucked.
  - with ktor, as everything is code (instead of annotations), there is a beautifully simple way to do it, and so extremely flexible, that every test strategy we can imagine can be implemented.

2.2.3. ADR - Persistence Technology

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- We need to choose a persistence technology for our application.
- A clear distinction between the needs for production vs test/development (local startup).
- We choose for a relational database…
  - No NoSQL store (MongoDB) or in-memory cache

Requirements:

Production DB:

- Available in corporate environment.
- If performance (searching/filtering) is an issue, maybe an elasticsearch index could be useful?

Test/Dev DB:

- Lightweight (in-memory)
- Fast startup and execution time
- No setup costs (installing, configuration, etc)

Options:

Production:

1. Oracle

2. Postgres/MySQL
3. MS SQL Server

Test/Dev:

1. H2
2. HSQLDB
3. Sqlite

Chapter 3. API Design

3.1. ADR - Filter ReST Resources

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- the user (via the frontend) needs to be able to filter bigger datasetes
- different fields have different types require different operators
  - general: equal, not equal
  - numeric: greater/lesser (or equal); between is done by adding both ;)
  - enum: (with)in
- operators are all in conjunctive form (coupled by AND)

Proposal:

Simple:

- `/crystals?filter=author eq 'Peter'`
- `/crystals?author=eq:Peter`
- `/crystals?weight=gt:3&weight=lte:10`
- `/crystals?state=in:FOO,BAR`

Complex:

- `/crystals?filter[0].property=owner.name&filter[0].operator=equals&filter[0].value=Peter`

Radical other approach would be to turn it into a POST (PUT?) request and provide a request body (full control). It feels like we are trying to squeeze a proper payload into query parameters (feels wrong, a smell we are misusing something).

Links:

- https://restfulapi.net/api-pagination-sorting-filtering/

3.2. ADR - Generate Custom OpenAPI Code

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Reason:

- We need to keep the API spec (YAML) and the code (Kotlin) in sync; we don't want to rely on doing this manually.
- Thus, establish an automatism via generation.
- We need something that fits our needs: modern, up2date, high quality code and libs.
- Current (Kotlin) generators are out of date (unusable), thus a custom generator is required.

Alternatives:

- Write some tests to verify sync of spec and code.
- No, as we want immediate feedback (compile time) about correct implementation.
- Use Java generators, they are more modern and Kotlin-Java interoperability works well.
- It would not clash as Java generated clients use different libraries (even when outdated).

Implementation requirements:

- Kotlin data classes with Kotlinx serialization annotations.
- Ability to hook specific type serializer (provided or custom).
- Think of `java.time.LocalDateTime` and enums.
- Generated separately (`view-model` subproject) from server routes; a unique requirement not provided by default generators.
- Interfaces for Ktor server.
- Some route providing interface types could be injected into Koin.
- Only bare skeleton mandatory (HTTP method and path), everything else optional (could go crazy with details).
- Client generation is optional.

3.2.1. Resources

- https://www.baeldung.com/java-openapi-custom-generator
- https://deepwiki.com/OpenAPITools/openapi-generator/5.1-creating-custom-generators

3.3. ADR - Paginate ReST Resources

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Approaches:

Offset:

- offset, limit; skip, take
- based upon it can implement page-based pagination (could support both actually)
- bad for big datasets; DB has to scan through rows

Others:

- Keyset pagination, seek method
- for example via `since_id` and a limit/take param
- only for IDs with auto-increment, or timestamps
- Time-based pagination
- for analytics/log monitoring only
- Cursor-based pagination
- like a bookmark, a backend-determined value not necessarily linked to any data fields (contrary to keyset)
- more complex to implement
- also return the "next cursor" in the response

Best Practices:

- clearly document pagination mechanism (openAPI, provide examples)
- don’t implement pagination for: A) small datasets and B) fast changing data
- use standard names
- provide page metadata:
- total pages, current page
- total items, items in page, size (request page size)
- has more
- provide navigation _links (HATEOES), next/previous/first/last
- either in metadata payload
- or in header: Link: <http://localhost:8080/api/books/paged?page=1&size=10>; rel="self"
- reuse test logic: each paginated endpoint needs to have the same set of tests

Questions:

- either use or at least get "inspired" by: https://github.com/perracodex/exposed-pagination
- how strict or lenient to be?
- negative numbers (skip -1, take 0)?
- take more than existing (page > totalPages)
- restrict a maximum take/limit param?

Resources:

- TODO get inspired by: https://github.com/perracodex/exposed-pagination
- https://www.merge.dev/blog/rest-api-pagination
- https://www.merge.dev/blog/api-pagination-best-practices
- https://restfulapi.net/api-pagination-sorting-filtering/

3.4. ADR - Secure HTTP Endpoints

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

How is authentification and authorization handled?

- IAM/LDAP
- Central security auth gateway in front
- service(s) only get an already checked user ID only (auth-agnostic)
- endpoints are configured centrally (?); the obligatory "auth matrix" ;)
- OAuth2 vs JWT?

3.5. ADR - Sort ReST Resources

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

About:

- call it sort or orderBy
- `orderBy=author,title`
- sort direction: asc/desc suffix
- `orderBy=author asc,title desc`
- or plus/minus: `orderBy=+author,-title`
- default to asc if not defined

Error Handling:

E.g.:

```
{
  "error": {
    "code": "InvalidOrderByExpression",
    "message": "The property 'author' in the orderby expression is not sortable",
```

```
    "details": [
      {
        "code": "UnsupportedSortProperty",
        "target": "author",
        "message": "Sorting by 'author' is not supported. Supported sort properties
are: ['id', 'title']"
      }
    ],
    "target": "orderby",
    "internal": {
      "trace-id": "00-d24f899d9c8a5a428fddd39399e7f58e-5c8a8949fcdd9a42-01",
      "timestamp": "2024-02-20T15:30:45.123Z",
      "request-id": "123e4567-e89b-12d3-a456-426614174000"
    }
  }
}
```

Links:

- https://restfulapi.net/api-pagination-sorting-filtering/

3.6. ADR - Versioning of HTTP Endpoints

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Options:

1. using header: `Accept: application/vnd.diamond.crystal.v1+json` and `Content-Type: …`
   - Keeps the URLs clean; metadata in headers
   - But not so obvious at first, requires additional logic to be implemented
2. encode in path: `GET /api/v1/crystals`
   - out-of-the-box supported by caching

Also:

- deprecate (TODO: tag OpenAPI spec as such, generate warning in code possible?!) and communicate
  - Keep track (logging) of usage to know when to turn off

Links:

- https://dev.to/abhivyaktii/api-versioning-using-headers-a-clean-and-flexible-approach-218p

Chapter 4. Application

Figure 2. A precious material

4.1. Tech Stack

- Gradle
- Kotlin
- Ktor
- Koin (not really DI, but a service locator, but ok ;))
- Kotest
- Exposed
- Liquibase
- Hoplite - typesafe loading application configuration (sys-props, env-vars, files)
- Arrow
  - Especially `Either` for better exception handling

4.2. ADR - Application Configuration

- Status: Draft
- Created: 1.12.2025
- Author: Seepick
- config as ENV vars or centralized config manager?

Links:

- https://softwarepatternslexicon.com/kotlin/microservices-design-patterns/configuration-management/

4.3. ADR - Layer Granularity

- Status: Draft
- Created: 5.12.2025
- Author: Seepick

Should the view-routing and view-controller-* modules be separate or merged?

- if testing strategy is testing from http (ktor test engine) then it can be merged.
  - if testing from controller (no ktor involved, plain code, simpler but less realistic/higher risk, require complementary tests and integration of those layers) then it should be separate.
- current situation feels like adding a costly feature without using its benefit
  - the modules are separated and the itests are not using the controllers (but HTTP ktor test engine)

Separate:

- very clean
  - view-route only purpose interact with web framework and leave
  - easy to test routes
- adds complexity (already heavy due to route - controller - domain - persist/extern)
  - controller interface, controller impl, register koin, ...
  - but enforces clean architecture!
- itests could use view-controllers instead using HTTP (from cucumber looks the same, just faster/easier setup)

Merged:

- faster development; less code/complexity
- unit (integration, as they use ktor) test for routes will be heavier.

4.4. Startup Arguments

- `java -jar diamond.jar printConfigOnly` - will print the read configuration object for verification
  - Used in `./local/test_apponfig.sh` to verify output

```
EnvConfig(
  ktor=KtorConfig(port=12),
  database=DatabaseConfig(
    jdbcUrl=db_url,
    username=db_user,
    password=****
  ),
  extern=ExternConfig(
    postsServiceBaseUrl=postsUrl,
    sftp=SftpConfig(
      remoteHost=sftpHost,
      port=22,
      username=sftpUser,
```

```
      authIsPassword=true,
      authPasswordOrPrivateKeyPath=****,
      knownHostsFilePath=knownHosts,
      strictHostChecking=true
    )
  )
)
```

4.5. Local Development

- run `nl.uwv.smz.diamond.app.LocalDiamondApp`
- Postman collection can be found at: `/local/Diamond.postman_collection.json`

IntelliJ plugins:

- Gradle
- EditorConfig
- detekt - configure with `/config/detekt.yml`
- ktlint
- kotest
- Markdown
- AsciiDoc
- plantuml4idea (graphivz/dot required)
- Gherkin
- Cucumber for Kotlin
- Cucumber for Java
- Cucumber+
- Cucumber Table Mapping
- kotlin-fill-class

4.6. Gradle

Sharing build logic via the `buildSrc` directory.

See: https://docs.gradle.org/current/userguide/sharing_build_logic_between_subprojects.html

4.6.1. Gradle Properties

Pass them either as `-P` (precendence) or `-D`.

- `diamond_version` - application version, defaults to `0` if not specified
  - displayed in info endpoint, used in generated documentation

- `runTestcontainersTests` - see task below
- `runEtests` - see task below

4.6.2. Gradle Tasks

- `./gradlew :app:buildFatJar -Pdiamond_version="42"` - build a standalone, executable Jar
- `./gradlew :app:generateConfigDoc` - custom task which generates the asciidoc for the config table (available env vars)
- `./gradlew :doc:SoftwareDocument:asciidoctorPdf` - generate PDF as `doc/SoftwareDocument/build/docs/asciidocPdf/index.pdf`
- `./gradlew :doc:SoftwareDocument:asciidoctor` - generate HTML to ``
- `./gradlew test -PrunTestcontainersTests=true` - run only the tests requiring testcontainers (docker; oracle)
- `./gradlew test -PrunEtests=true` - run the Karate tests located in the `etest` subproject
- `./gradlew dependencyUpdates` - to check for outdated dependencies (help category, using manes plugin)
- `./gradlew publishImageToLocalRegistry` or `runDocker` (or `publishImage` once remote registry is configured)

4.7. ADR - Manage Dependencies

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Which way to properly manage gradle dependencies?

Problem description:

- versions repeatedly being declared, danger of mismatch (same library, different versions)
  - especially true for multi-module projects
  - leading to runtime error due to binary incompatibility (NoSuchMethodError, etc.)
- cumbersome as in lots to type and repeat (we want to be concise and fast)

Requirements:

- not abstracting/hiding too much (e.g. one god dependency), but be explicit/transparent enough for understanding
  - be concise enough, while capturing the essence for clarity (intuitive comprehension) sake
- not too complex, no over-engineering, just get it "good enough"
- deal with a huge amount of versions/dependencies/plugins
- custom plugin declaration (to share common code as extensions/compositions) can use it as well

Alternatives:

Either: 1) buildSrc custom code or 2) use gradle's standard version catalog.

Custom `buildSrc` Code:

Pros:

- it's plain kotlin code with all its advantages
  - refactoring safe
  - immediate feedback from compiler if something is wrong
  - go to source with CTRL+click (javadoc)
- full control and flexibility (custom made solution)
  - e.g. group versions and dependencies
- already used for custom plugins
  - BUT: it is NOT possible to use things from the catalog for those

Cons:

- it's custom made for something a default solution already exists
- doesn't support view usages

4.7.1. Gradle Version Catalog

- https://docs.gradle.org/current/userguide/version_catalogs.html

Pros:

- it's a out-of-the-box supported solution, people know it, well documented
- allows for grouped dependencies, so-called libraries

Cons:

- TOML syntax is not as useful as code (auto-completion, type feedback, etc.)
- going into source implementation/definition not possible (CTRL+click), cumbersome
- if changing/refactoring, then step-by-step errors by each build, cumbersome

```
[bundles]
ktor = ["ktor-core", "ktor-json", "ktor-foobar"]

[plugins]
short-notation = "some.plugin.id:1.4"
versions = { id = "com.github.ben-manes.versions", version.ref = "manes-versions" }
```

Conclusion:

- both support grouping and a nested-hierarchy to control a vast amount of declarations
- both support auto-completion
- custom plugin declaration is fucked
    - neither the custom code approach, nor the version catalog supports it (too early in build phase i assume)

4.8. ADR - Use Gradle Kotlin DSL

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context: * obviously not groovy (cumbersome) * as project is already in kotlin, makes sense to configure gradle also with kotlin * has become stable enough the last years (support, plugins/configuration, sufficient documentation on the internet)

4.9. Documentation

4.9.1. ADR - Documentation Tool

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Options:

- Markdown
    - Well known and well supported
    - Might not provided sufficient functionality for a more complex/bigger documentation project
- Asciidoc
    - Somewhere between Markdown and LaTeX in usability/support and functionality/complexity
    - GitHub can also render it, just like markdown, yay
- LaTeX
    - Great for writing books, but definitely poses a too high barrier of entry for people unfamiliar with it
    - Local latex installation is far more than needed compared to provided infrastructure for other technologies

4.9.2. ADR - Diagram Drawing Tool

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Requirements:

- programmatic (source code, version trackable)
- but high barrier entry
- no WYSIWYG, thus not suitable for complex (and "pretty") diagrams
- defined along adoc
- easy access (no visio)
- user/rights/auth
- web based

Options:

- plantuml
- mermaid
- ...?

PlantUml:

Install `dot` binary from https://graphviz.org/download/ to be able to build diagrams with code.

```
[plantuml,inlineUml,svg]
....
@startuml
[Inline] --> Interface1
[Inline] -> Interface2
@enduml
....

Or included:

[plantuml,includedUml,svg]
....
include::includes/diagram.puml[]
....
```

- With markdown (GitHub) also PlantUML possible "somehow", but not really...
  - https://gist.github.com/noamtamim/f11982b28602bd7e604c233fbe9d910f

Chapter 5. Coding

Kotlin.

- list the bad things as well; honesty, expection mgmt, credibility.
- intellij support; crash; young

5.1. Coding Philosophy

- fail early, fail fast
- e.g. if application config something is configured wrong/missing
- validate data (front incoming HTTP, back external systems/DB, immediately throw)
- code first (no strings/annotations, no declarative xml/json/yaml)
- no annotations, strings, properties, yamls… just code
- be in control
- no classpath scanning, reflection, other look-ups, no auto-magically something
- avoid intrusive frameworks (the systems serves us, we don’t serve the system)
- functional
- pure functions: stateless, side-effect free
- code as named expressions
- prefer single-expression-method `fun foo() = doSome().doOther().also { done(it) }`

5.1.1. Coding Principles

- instead of regular methods → "named expressions"
  - always use "=" after method signature; single expression body
  - no state, no "val" keyword, all just immediate execution (more functional)
  - forces to write very short methods, easy to read and comprehend
  - methods have names, they create logical structure of arbitrary code

```
fun `BAD single expression`(): String = "foo"

fun `GOOD single expression`() = "foo"

fun `BAD single factory expression`(): FooDto = FooDto(
    id = 42,
)

fun `GOOD single factory expression`() = FooDto(
    id = 42,
)
```

```
fun `BAD db access`() {
    transactional {
        repo.update()
    }
}

fun `GOOD db access`() = transactional {
    repo.update()
}

fun Service.`BAD wither`(): Service {
    setFoo("bar")
    return this
}

fun Service.`GOOD wither`() = apply {
    setFoo("bar")
}
```

5.2. ADR - Generating Sources

- Status: Draft
- Created: 6.12.2025
- Author: Seepick

Context:

- We use specifications to document the contract, provided by ourselves and third parties.
- Primarily OpenAPI YAML and WSDL XML files.

> Should we regenerate sources each time and host the code next to our own, or should be pre-generate and store them in a dedicated repository?

Options:

Re-generate:

- GOOD: Everything is self-contained; simple, easy.
- BAD: Slows down the build process due to regeneration despite no API changes (caching?).
- BAD: Exclusion configuration necessary for static code analysis, coverage, etc. (different rules apply to them)

Pre-generate:

- GOOD: Faster build times.

- GOOD: A change of the API contract must be a conscious act
  - Leads to more safety/stability
  - But it’s also a BAD argument: slower development time, more complex!
- For 3rd party APIs, we would expect a given SDK anyways… so we do it ourselves.

Decision:

- Pre-generate!
- Divide and conquer: Manage the bigger complexity by outsourcing.
- Split things apart, separation of concerns.

5.3. Libraries

5.3.1. ADR - Database Access Library

- Status: Accepted
- Created: 1.12.2025
- Author: Seepick

Context:

- We need a way to access relational database from Kotlin code.
  - We talk about DB access (as in JDBC), not a full ORM solution.
- Question is, which library to use?

Requirements:

- Needs to be type-safe (no "stringly-typed" expressions)
- Little/no code generation magic
- Kotlin-idiomatic (coroutines, DSLs)
  - For using coroutines we would need a non-blocking implementation (no JDBC)

Options:

1. Exposed by JetBrains
   - Seems production ready; mature and stable enough
   - Lightweight and typesafe
2. KTorm
   - ORM mapper for Kotlin, pure JDBC
3. JPA / Hibernate
   - Heavyweight, not Kotlin-idiomatic
   - Lots of magic, code generation, proxies, etc.

Choice:

- Exposed
- It fits with the rest of the used ecosystem.
- We use the conventional DSL, not the DAO API (seem unfinished).

Example of the DAO API:

```
// definition
class CrystalDaoEntity(id: EntityID<UUID>) : UUIDEntity(id) {
    var weightInGrams by CrystalTable.weightInGrams
    companion object : UUIDEntityClass<CrystalDaoEntity>(CrystalTable)
}
// usage
CrystalDaoEntity.findById(id)
CrystalDaoEntity.new(UUID.randomUUID()) {
    weightInGrams = 42
}
CrystalDaoEntity.findByIdAndUpdate(id) {
    it.weightInGrams = 42
}
```

5.3.2. ADR - Scheduler Library

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- we need to run jobs regularly in a cronjob way
- no spring scheduler available

Options:

- Quartz https://www.quartz-scheduler.org
- enterprise (heavy, slow) scheduler; the "platzhirsch", used to be the one and only; decades
- but the pros comes with cons: outdated, verbose APIs
- features: persisted in DB (requires 10+! tables), retry, clustering, prioritization etc
- no monitoring (grafana, jaeger)
- JobRunr https://www.jobrunr.io
- lightweight, modern; RDBMS (and NoSQL)
- features: distributed, dashboard, retry
- db-scheduler https://github.com/kagkarlsson/db-scheduler

- lightweight: performant, minimal deps; still offer what's need: persistence, cluster

maybe there is a mature enough kotlin library which leverages coroutines?

- KtScheduler https://github.com/Pool-Of-Tears/KtScheduler
- seems like a small hobby project…

Links:

- https://www.jobrunr.io/en/blog/2024-10-31-task-schedulers-java-modern-alternatives-to-quartz/

5.3.3. ADR - Access SFTP service

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- JSch is very old and very outdated
- Use forked and updated `com.github.mwiede:jsch` instead of original `com.jcraft:jsch`

5.3.4. ADR - DateTime library

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

> Use java-time or kotlin-time?

Java:

- GOOD: robust, well-known, well-integrated with libs (serialization, DB)
- BAD: cumberesome
- BAD: not supported by kotlin-idiomatic libs

Kotlin:

- GOOD: lightweight, probably/most-likely kotlin idiomatic; modern API
- GOOD: integration with kotlinx-serialization and exposed
- BAD: not all (java) libs support it
- BAD: higher barrier of entry for others
- any benefit from its multiplatform nature?

Conclusion:

- unstable API is too dangerous for production!
- NoSuchMethod error, incompatible versions on classpath
- kotlin is not well integrated yet (kotest)

5.3.5. ADR - Logging with Kotlin

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Logging evolution:

```
if (log.isDebugEnabled()) {
    log.debug("some message " + message + " was returned")
}
```

This way we only create the string if it is really necessary, bad. But it's very verbose, to do it correct and concise; bad.

Then we switched to a `String.format`-like approach to fix this, to always be able to invoke the log method without performance impact:

```
log.debug("some message %s was returned", message)
```

This two issues (conciseness, performance) but still was not readable, especially when logging longer messages with multiple values. The displacement-nature of these kind of formatting approaches is just not doing it…

Finally we are doing it nicely, using Kotlin's string templates which are basically string concatenations, and the ease of lambda declarations:

```
log.debug { "some message $dto was returned" }
```

Logger Declaration:*

Thanks to `io.github.oshai:kotlin-logging-jvm` (former "mu-kotlin") the declaration of a logger is as simple as it can be:

```
private val log = logger {}
```

It will look up the surrounding class and thus use the right logger factory itself. It uses Slf4j underneath.

The actual logging implementation is done by Logback (the better Log4j).

5.3.6. ADR - Logging Configuration

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- following principle: code > configuration (yaml/properties)
  - more flexibility, more control, more sophisticated, more robust
- programmatic logging configuration
- dev/test different from default-prod
- provide mechanism to override(?), maybe even
- TODO: runtime, to "debug" = increase log; in running prod

Chapter 6. Testing

6.1. Test Types

- unit tests
- integration tests
- application/module/component tests
- acceptance/functional tests
- contract tests
- external system tests (backend)
- system tests
- end-to-end tests

other types:

- ATDD, BDD Tests
- smoke tests
- regression tests
- performance/load/stress tests (gatling or jmeter)
- security/penetration tests

other other types:

- sanity tests
- exploratory tests
- usability tests
- compatibility tests
- localization/internationalization tests

6.1.1. End-to-End

- in real world, it would be in a separate GIT repo
- use karate, only change base URL
- use @WIP tagging; maybe @JIRAID-12345
- use @ReadOnly tagging to execute target production as well

6.2. Best Practices

Watch out to not test same multiple times, we need to be able to still move and change fast!

6.3. Test Tech

6.3.1. ADR - Use BDD

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- TDD on unit test level; BDD for integration tests (on higher acceptance/functional level)
- using a gherkin-based format; use cucumber
- when having both plugins (cucumber and karate) installed, configure file types to be different
  - cucumber: *.feature
  - karate: *.karate.feature

6.3.2. ADR - Use Cucumber Lambdas

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- use more kotlin-idiomatic approach by the java8 lambda functionality
  - instead the old fashioned annotationstyle
- both approaches are fully supported by the intellij-plugin
  - click into declaration possible from feature files

Links:

- https://github.com/cucumber/cucumber-jvm/blob/main/cucumber-java8/README.md
- Example: https://github.com/cucumber/cucumber-jvm/blob/main/cucumber-kotlin-java8/src/test/kotlin/io/cucumber/kotlin/LambdaStepDefinitions.kt

6.3.3. ADR - Verify Coverage

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- Goal: automatically verify minimum test coverage threshold

    - Fail build if not achieved
    - Provide dashboards (Sonarqube) on current state and changes (delta)
- coverage 100%?, kotlin inlining issue…

Options:

- jacoco
    - very mature, well supported by tools
    - other coverage tools usually can export to jacoco's format too
    - not supporting Kotlin's inline
- kover
    - specific for kotlin, from jetbrains
    - bleeding edge (child sicknesses?)

6.3.4. ADR - Assert JSON

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- for itest with cucumber, we need a library which supports something like: `$.items[3].title eq 'foobar'`
- a library which we can be passed a complex string to evaluate
- for complete JSON comparison, use: https://github.com/skyscreamer/JSONassert

Options:

- assert full JSON: https://github.com/skyscreamer/JSONassert
- regular expression support: https://fslev.github.io/json-compare/
- jayway path is an option (`com.jayway.jsonpath:json-path`)
- it provides hamcrest matchers `com.jayway.jsonpath:json-path-assert`
- tutorial: https://www.baeldung.com/guide-to-jayway-jsonpath

Chapter 7. Code Quality

- quality gates
- sonarqube and kotlin tools
- CAVE: there is much overlap in the tools
  - test coverage enforcing by gradle (kover) and sonarqube
  - lots with detekt/ktlint (intellij? sonarqube?)

7.1. Static Code Analysis

Detekt

ktlint

- https://pinterest.github.io/ktlint/1.8.0/

7.2. SonarQube

- use the free tier for open source projects
  - See: https://sonarcloud.io/project/configuration/AutoScan?id=seepick_diamond
- define new code by days (continuous delivery), not by change delta (planned releases)
- use it to report coverage (enforced by gradle-kover itself, not sonar)

7.3. ADR - Static Code Analysis

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- actually also runtime detection possible!
- also ktlint
- enable `Refactor → AutoCorrect by detekt rules`
- bad: e.g. line length not auto-configured in intellij by detekt rule :-(

7.4. ADR - Enforce Failure Handling

- Status: Draft
- Created: 1.12.2025
- Author: Seepick

Context:

- Pure Functions:
  - return values are identical for identical arguments (no variation with static variables, non-local variables, mutable reference arguments or input streams, i.e., referential transparency)
  - the function has no side effects (no mutation of non-local variables, mutable reference arguments or input/output streams)
- Throwing (unchecked) exceptions is a sort of an implicit return value which can be forgotten to be handled properly, resulting in false-500 responses.
- Use the Arrow library to introduce a more functional approach, the `Either` class specifically
  - TODO: in the future maybe also optics to manipulate deep nested, immutable data classes.

Either:

- Good:
  - developers need to think (forced to, explicit) error types, mapping of them (translation of corrupt data to bad request/internal error)
  - supports DDD by simulating failure-aware constructors (companion operator fun with Either return type)
- Bad:
  - sometimes not clear if not familiar with it
  - sometimes IDE doesn't suggest `.bind()` although it's the solution

Chapter 8. DevOps

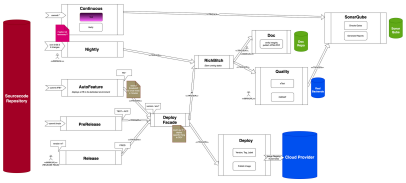

Figure 3. Sketching a build pipeline overview

8.1. Environment Variables

Param	Type	Default	Description
DATABASE_JDBCURL	string	-	JDBC driver URL
DATABASE_PASSWORD	string	-	DB password
DATABASE_USERNAME	string	-	DB username
EXTERN_POSTSSERVICEBASEURL	string	-	Base URL for the external posts service
EXTERN_SFTP_AUTHISPASSWORD	boolean	-	Whether using password or private key.
EXTERN_SFTP_AUTHPASSWORDORPRIVATEKEYPATH	string	-	Either password or private key.
EXTERN_SFTP_KNOWNHOSTSFILEPATH	string	-	Path to SSH known hosts file.
EXTERN_SFTP_PORT	integer	22	Port of the SFTP server.
EXTERN_SFTP_REMOTEHOST	string	-	Host of the SFTP server.
EXTERN_SFTP_STRICTHOSTCHECKING	boolean	true	Disable security check; do NOT activate in production; pretty please.
EXTERN_SFTP_USERNAME	string	-	Login username.
KTOR_PORT	integer	8080	Webserver HTTP port to listen to

8.2. Distributed Services

Their specifics require:

- Versioning, backwards compatiability.
- Resilience and Fault Tolerance: circuit breakers, retries, and bulkheads.

Chapter 9. Appendix

9.1. Resources

- ADR: https://github.com/joelparkerhenderson/architecture-decision-record
- SFTP inspirations:
  - https://github.com/alkaphreak/marstech-sftp/blob/main/src/main/kotlin/fr/marstech/mtsftp/service/SftpFileConnectorServiceImpl.kt
  - https://medium.com/whozapp/sftp-test-implem-of-jsch-with-kotlin-testcontainers-and-spring-boot-native-537f624da895
  - https://github.com/testcontainers/testcontainers-java/blob/main/examples/sftp/src/test/java/org/example/SftpContainerTest.java
- Exposed:
  - DAO: https://ktor.io/docs/server-integrate-database.html#create-mapping
- Cucumber:
  - https://www.baeldung.com/java-cucumber-hooks
  - https://towardsdev.com/how-to-perfectly-test-your-ktor-application-94fb92c5a303
  - https://www.baeldung.com/cucumber-data-tables
  - https://github.com/cucumber/cucumber-jvm/tree/main/datatable